The Journal of Systems and Software 234 (2026) 112690

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

MENTOR: Fixing introductory programming assignments with formula-based fault localization and LLM-driven program repair

Pedro Orvalho [a,1,*], Mikoláš Janota [b], Vasco Manquinho [c]

[a] Department of Computer Science, University of Oxford, Oxford, United Kingdom
[b] CIIRC, Czech Technical University in Prague, Prague, Czech Republic
[c] INESC-ID, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal

ARTICLE INFO

Editor: Dr Antonio Filieri

Keywords:
Automated program repair
Program analysis
Formula-based fault localization
LLM-driven program repair
Computer-aided education

ABSTRACT

The increasing demand for programming education has led to online evaluations like MOOCs, which rely on introductory programming assignments (IPAs). A major challenge in these courses is providing personalized feedback at scale. This paper introduces MENTOR, a semantic automated program repair (APR) framework designed to fix faulty student programs. MENTOR validates repairs through execution on a test suite, and returns the repaired program or highlights faulty statements.

Unlike symbolic repair tools like CLARA and VERIFIX, which require correct implementations with identical control flow graphs (CFGs), MENTOR's LLM-based approach enables flexible repairs without strict structural alignment. MENTOR clusters successful submissions regardless of CFGs, and employs a Graph Neural Network (GNN)-based variable alignment module for enhanced accuracy. Next, MENTOR's fault localization module leverages MaxSAT techniques to pinpoint buggy code segments precisely. Finally, MENTOR's program fixer integrates Formal Methods (FM) and Large Language Models (LLMs) through a Counterexample Guided Inductive Synthesis (CEGIS) loop, iteratively refining repairs. Experimental results show that MENTOR significantly improves repair success rates, achieving 64.4 %, far surpassing Verifix (6.3 %) and Clara (34.6 %). By merging formula-based fault localization, and LLM-driven repair, MENTOR provides an innovative, scalable framework for programming education.

1. Introduction

Every year, thousands of students enroll in programming-oriented courses. With the rapid growth of Computer Science courses, providing personalized and timely feedback on *introductory programming assignments* (IPAs) and software projects has become a significant challenge, requiring substantial time and effort from faculty (Gulwani et al., 2018).

Automated Program Repair (APR) has emerged as a promising solution to this challenge, aiming to deliver automated, comprehensive, and personalized feedback to students about their programming errors (Gulwani et al., 2018; Ahmed et al., 2022; Wang et al., 2018; Hu et al., 2019; Orvalho et al., 2024c). Traditional semantic APR techniques based on *Formal Methods* (FM), while providing high-quality fixes, are often slow and may struggle when the correct implementation diverges significantly from the erroneous one (Contractor and Rivero, 2022; Orvalho et al., 2025b). These APR approaches do not guarantee minimal repairs, as they align an incorrect submission with a correct implementation for the same IPA. If the alignment is not possible, these tools return a structural mismatch error, leaving the program unrepaired. In the past decade, there has been a surge in Machine Learning (ML) techniques for APR (Gupta et al., 2017; Mesbah et al., 2019; Gupta et al., 2019; Yasunaga and Liang, 2020; Rolim et al., 2017; Bhatia et al., 2018; Orvalho et al., 2023). ML-based approaches require multiple correct implementations to generate high-quality repairs, and need considerable time and resources to train on correct programs. While these approaches generate repairs more quickly, they often produce imprecise and non-minimal fixes (Wang et al., 2018).

More recently, *Large Language Models* (LLMs) trained on code (LLMCs) have shown great potential in generating program fixes (Joshi et al., 2023; Xia et al., 2023a; Jin et al., 2023; Wei et al., 2023; Fan et al., 2023; Xia et al., 2023b; Zhang et al., 2024; Phung et al., 2023). LLM-based APR can be performed using zero-shot learning (Xia and Zhang, 2022), few-shot learning (Zhang et al., 2024) or fine-tuned models (Jin et al., 2023). Fine-tuned models are the most commonly used, where the model is trained for a specific task. In contrast, zero-shot learning refers to the ability of a model to correctly perform a task without having seen

* Corresponding author.
E-mail addresses: pedro.orvalho@cs.ox.ac.uk (P. Orvalho), mikolas.janota@cvut.cz (M. Janota), vasco.manquinho@tecnico.ulisboa.pt (V. Manquinho).
[1] Part of this work was conducted at INESC-ID, Instituto Superior Técnico, U. Lisboa.

https://doi.org/10.1016/j.jss.2025.112690
Received 16 June 2025; Received in revised form 30 September 2025; Accepted 27 October 2025
Available online 10 November 2025
0164-1212/© 2025 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Table 1
Test-suite with three tests (t_0, t_1, and t_2). Given as input three numbers (num1, num2, and num3), each test should return the value in its respective output column.

	Input			Output
	num1	num2	num3	
t_0	1	2	3	3
t_1	6	2	1	6
t_2	-1	-2	-3	-1

any examples of that task during training. Few-shot learning refers to the LLMs's ability to perform tasks correctly with only a small number of examples provided. Furthermore, the ability to generalize using zero or few-shot learning enables LLMs to handle a wide range of tasks without the need for costly retraining or fine-tuning. Nonetheless, few-shot learning can lead to larger fixes than necessary, as it is based on a limited number of examples. Moreover, LLMs do not guarantee minimal repairs and typically rewrite most of the student's implementation to fix it, rather than making minimal adjustments, making their fixes harder for students to learn from.

In this work, we present MENTOR, a clustering-based semantic program repair tool for the C programming language capable of providing automated feedback for introductory programming exercises. MENTOR leverages past student submissions to assist in repairing incorrect code. It combines *Maximum Satisfiability* (MaxSAT)-based fault localization with large language models (LLMs), guiding them using bug-free program sketches. MENTOR's main objective is to repair buggy programs using correct implementations, even when their control-flow graphs (CFGs) differ, a capability not supported by state-of-the-art semantic repair tools.

1.1. Problem description

Consider the program presented in Listing 1, which aims to determine the maximum among three given numbers. Based on the test suite shown in Table 1, the program is buggy as its output differs from the expected results. The set of minimal faulty lines in this program includes lines 4 and 8, as these two `if` conditions are incorrect. A good way to provide personalized feedback to students on their IPAs is to highlight these two buggy lines. However, it is essential to check these faults by fixing the program and evaluating the repaired program against the test suite.

Using traditional Automated Program Repair (APR) tools for introductory programming assignments (IPAs) based on Formal Methods, such as CLARA (Gulwani et al., 2018) or VERIFIX (Ahmed et al., 2022), the program in Listing 1 cannot be fixed within 90 seconds. CLARA takes too long to compute a 'minimal' repair by considering several correct

```
1  int main(){// finds max of 3 numbers
2     int f,s,t;
3     scanf("%d%d%d",&f,&s,&t);
4     if (f < s && f >= t)//fix: f >= s
5        printf("%d",f);
6     else if (s > f && s >= t)
7        printf("%d",s);
8     else if (t < f && t < s)
9     // fix: t > f and t > s
10       printf("%d",t);
11
12    return 0;
13 }
```

Listing 1. Semantically incorrect program. Faulty lines: {4,8}.

```
1  int main() {
2     int m1,m2,m3,m;
3     scanf("%d%d%d",&m1,&m2,&m3);
4     m = m1 > m2 ? m1 : m2;
5     m = m3 > m ? m3 : m;
6     printf("%d\n", m);
7
8     return 0;
9  }
```

Listing 2. Reference implementation.

```
1  int main(){
2     int f,s,t;
3     scanf("%d%d%d",&f,&s,&t);
4     @ HOLE 1 @
5        printf("%d",f);
6     else if (s > f && s >= t)
7        printf("%d",s);
8     @ HOLE 2 @
9        printf("%d",t);
10
11    return 0;
12 }
```

Listing 3. Program sketch with holes.

implementations for the same IPA, while VERIFIX returns a compilation error. Conversely, using state-of-the-art LLMs trained for coding tasks (LLMCs), GRANITE (Mishra et al., 2024) or CODEGEMMA (Zhao et al., 2024), would involve providing the description of the programming assignment and some examples of input-output tests. Even with these features, neither LLM could fix the buggy program in Listing 1 within 90 seconds when repeatedly testing and refining their fixes. If the lecturer's reference implementation shown in Listing 2 is suggested as a reference in the prompt, both LLMs simply copy the correct program, ignoring instructions not to do so.

Hence, symbolic approaches demand an excessive amount of time to produce an answer, and LLMs, while fast, often produce incorrect fixes. Thus, MENTOR aims to address the limitations of current state-of-the-art APR frameworks for IPAs by combining the strengths of both approaches. State-of-the-art Maximum Satisfiability (MaxSAT)-based Fault Localization (Orvalho et al., 2024b) can rigorously identify buggy statements, which can then be highlighted in the LLM prompt to focus on the specific parts of the program that need fixing. Listing 3 shows an example of a program sketch, which is a partially incomplete program where each buggy statement from the original incorrect program in Listing 1 is replaced with a `@ HOLE @`. Instructing the LLMs to complete this program allows both GRANITE and CODEGEMMA to fix the buggy program in a single interaction, returning the program in Listing 4.

Thus, MENTOR aims to repair a student's incorrect submission without relying on a correct solution with the same program structure, while also minimizing changes to the student's code, similar to symbolic approaches (Gulwani et al., 2018; Ahmed et al., 2022).

1.2. Paper overview and contributions

Section 2 presents the basic definitions and notation used throughout this paper. Next, Section 3 describes the architecture of MENTOR, which consists of four main modules: (1) program clustering, (2) variable alignment, (3) fault localization, and (4) program fixer. MENTOR employs state-of-the-art tools at each stage: INVAASTCLUSTER (Orvalho et al.,

```c
int main(){
   int f,s,t;
   scanf("%d%d%d",&f,&s,&t);
   if (f >= s && f >= t)
      printf("%d",f);
   else if (s > f && s >= t)
      printf("%d",s);
   else
      printf("%d",t);

   return 0;
}
```

Listing 4. GRANITE's fix using the program sketch.

2025b) for clustering, a GNN-based approach (Orvalho et al., 2023) for variable mapping, and CFAULTS (Orvalho et al., 2024b) for formula-based fault localization. Furthermore, Section 4 describes our experimental results on C-PACK-IPAS Orvalho et al. (2024a), a benchmark of IPAs. These experiments demonstrate that MENTOR's hybrid repair method, integrating Formal Methods (FM)-based fault localization with LLMs, significantly enhances repair success rates while yielding smaller, more precise fixes. This innovative approach outperforms other existing repair strategies and state-of-the-art symbolic tools. For instance, VERIFIX repairs only 6.3 % of C-PACK-IPAS, and CLARA achieves a repair rate of 34.6 %. In contrast, depending on the prompt configuration and the specific LLM utilized, MENTOR achieves impressive repair rates, ranging from 37.3 % to 64.4 %. Finally, Section 5 reviews related work, and the paper concludes in Section 6.

The main novelty of MENTOR, compared to our earlier work (Orvalho et al., 2025a), lies in its program fixer module, which integrates the outputs of three key components: (1) cluster representatives, (2) variable mappings, and (3) localized faulty statements. These elements are incorporated into structured prompts that drive an LLM-based, counterexample-guided program repair process. Furthermore, in addition to the research questions explored in our earlier work, this paper introduces several new dimensions of analysis that further advance the understanding of LLM-based program repair. Specifically, in Section 4, we investigate: (i) the usefulness of providing LLMs with a variable mapping between a buggy program and a correct implementation of the same IPA (RQ6); (ii) the effectiveness of MENTOR in repairing programs that CLARA fails to address due to control-flow mismatches (RQ7); (iii) how LLM repair performance is influenced by varying levels of program complexity (RQ8); and (iv) the number of CEGIS iterations typically required by LLMs to repair programs, and what this reveals about their repair efficiency (RQ9). These additional research questions broaden the scope of our study and provide new insights into the capabilities and limitations of LLM-based automated program repair.

In summary, this work presents MENTOR, a semantic automated program repair framework, for the C programming language, designed to provide automated feedback in IPAs. This paper makes the following contributions:

- MENTOR addresses the Automated Program Repair (APR) problem through an LLM-driven Counterexample-Guided Inductive Synthesis (CEGIS) approach;
- MENTOR employs MaxSAT-based Fault Localization to guide and minimize LLMs' patches to incorrect programs by feeding them bug-free program sketches.
- MENTOR combines state-of-the-art modules for program clustering (Orvalho et al., 2025b), variable alignment (Orvalho et al., 2023), and MaxSAT-based fault localization (Orvalho et al., 2024b); within an LLM-driven program fixer that orchestrates the repair process.
- Experimental results show that our approach enables all six evaluated LLMs to fix more programs and produce smaller patches compared to alternative configurations and symbolic tools.
- All code and experiments will be made publicly available on GitHub: https://github.com/pmorvalho/MENTOR, and on Zenodo (Orvalho et al., 2025c).

2. Preliminaries

This section provides definitions used throughout this work.

Maximum satisfiability (MaxSAT). The *Boolean Satisfiability* (SAT) problem is the decision problem for propositional logic (Biere et al., 2009). A propositional formula in Conjunctive Normal Form (CNF) is a conjunction of clauses where each clause is a disjunction of literals. The *Maximum Satisfiability* (MaxSAT) problem is an optimization version of SAT, i.e., the goal is to find an assignment that maximizes the number of satisfied clauses in a formula (Bacchus et al., 2021).

Basic block (BB). A *basic block* is a linear sequence of program instructions having one entry point (the first instruction executed) and one exit point (the last instruction executed). It may, of course, have many predecessors and many successors and may even be its own successor. Program entry blocks might not have predecessors that are in the program; program terminating blocks never have successors in the program (Allen, 1970).

Control dependence. A *control dependence* occurs between a statement and a control predicate whose value dictates if the statement is executed or not (Ferrante et al., 1987).

Control flow path. A *control flow path* is defined by the order in which program statements, instructions, and function calls are executed.

Control flow Graph (CFG). A *control flow graph (CFG)* is a directed graph in which the nodes represent basic blocks, and the edges represent control flow paths (Allen, 1970).

Program. A *program* is considered sequential, comprising standard statements such as assignments, conditionals, loops, and function calls, each adhering to their conventional semantics in C. A program P is deemed to contain a bug when an assertion violation occurs during its execution with input I. Conversely, if no assertion violation occurs, the program is considered correct for input I. In cases where a bug is detected for input I, it is possible to define an error trace, representing the sequence of statements executed by program P on input I (Orvalho et al., 2024b).

Abstract syntax tree (AST). An AST is a syntax tree in which each node represents an operation, and the node's children represent the arguments of the operation for a given programming language described by a Context-Free Grammar. An AST depicts a program's grammatical structure (Aho et al., 1986).

Synthesis problem. For a given program's specification S (e.g., input-output examples), G a context-free grammar, and O be the semantics for a particular Domain-specific language, the goal of *program synthesis* is to infer a program P such that (1) the program is produced by G, (2) the program is consistent with O and (3) P is consistent with S (Orvalho et al., 2019, 2020; Silva, 2019).

Please refer to Aho et al. (1986) for more context about context-free grammars (CFGs), and Domain-specific language (DSL).

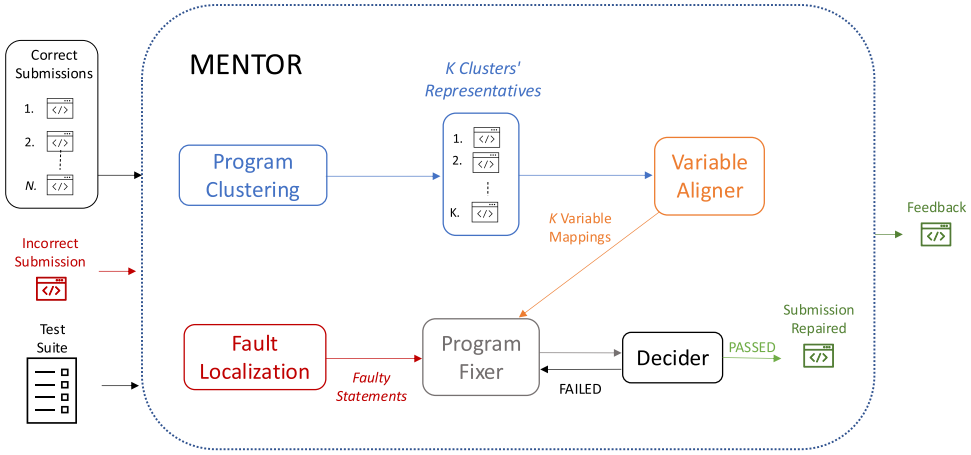


Fig. 1. Overview of MENTOR.

Semantic program repair. Given (T, G, O, P), let T be a set of input-output examples (test suite), G be a grammar, O be the semantics for a particular Domain-specific language, and P be a syntactically well-formed program (i.e., sets of statements, instructions, expressions) consistent with G and O but semantically erroneous for at least one of the input-output tests (i.e., $\exists(t^i_{in}, t^i_{out}) \in T \ : \ P(t^i_{in}) \neq t^i_{out}$).

The goal of *Semantic Program Repair* is to find a program P_f by semantically change a subset S_1 of P's statements ($S_1 \subseteq P$) for another set of statements S_2 consistent with G and O, such that,

$$P_f = ((P \setminus S_1) \cup S_2)$$

and

$$\forall(t^i_{in}, t^i_{out}) \in T \ : \ P_f(t^i_{in}) = t^i_{out}.$$

Moreover, automated program repair tools typically perform program synthesis to synthesize the set of program statements S_2 needed to fix the erroneous program P (Orvalho et al., 2019; da Silva, 2025).

3. MENTOR

This section presents MENTOR, a semantic automated program repair framework designed to provide automated feedback in introductory programming assignments (IPAs).

The overall architecture of MENTOR is shown in Fig. 1. MENTOR receives as input an incorrect submission for a given introductory programming assignment (IPA), a test suite, and a set of N correct submissions for the same IPA. MENTOR is divided into five main modules: (1) program clustering, (2) variable aligner, (3) fault localization, (4) program fixer, and (5) decider. MENTOR starts by clustering, using InvAASTCluster (Orvalho et al., 2025b), all correct submissions using these program's sets of invariants and abstract syntax trees (ASTs) as described in Section 3.1. Next, MENTOR employs Graph Neural Networks (GNNs) to map the set of variables between each cluster's representative and the incorrect submission (Orvalho et al., 2023, 2022b) based on the ASTs of both programs (see Section 3.2). Additionally, MENTOR uses CFaults (Orvalho et al., 2024b), a state-of-the-art formula-based fault localization (FBFL) technique for C programs capable of addressing multiple faults. CFaults, described in Section 3.3, leverages Model-Based Diagnosis (Reiter, 1987) with multiple observations, integrating all failing test cases into a single MaxSAT formula to ensure consistency in the fault localization process.

The program fixer module integrates all the features computed so far: (1) cluster representatives, (2) variable mappings, and (3) localized faulty statements. These features are incorporated into prompts to guide our LLM-driven counterexample guided program repair process (Orvalho et al., 2025a), described in Section 3.4. Finally, in the decider module, MENTOR checks if the candidate program proposed by the program fixer is correct by evaluating it against the provided test suite. If the candidate program remains incorrect, the program fixer generates a new candidate. Otherwise, MENTOR returns personalized feedback to students, either by providing the fixed program or by highlighting the buggy statements in their code.

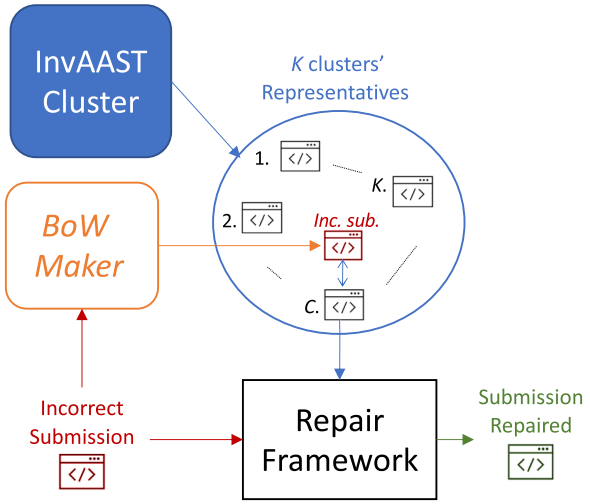
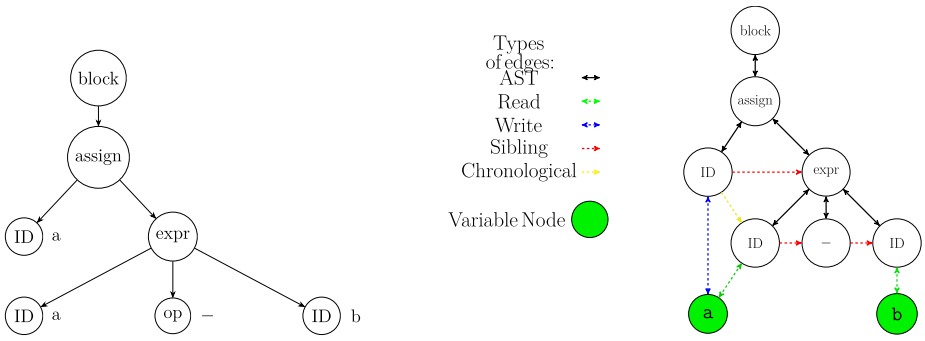


Fig. 2. Finding the closest correct program, i.e., the closest correct program representative to the incorrect submission vector representation.

3.1. Program clustering

To support the repair process, MENTOR aims to find a correct implementation for the same IPA that is structurally and semantically close to a given buggy submission. Instead of exhaustively comparing all correct submissions, MENTOR leverages InvAASTCluster (Orvalho et al., 2025b), a state-of-the-art clustering tool for IPAs. InvAASTCluster groups correct programs based on their sets of program invariants and their *anonymized abstract syntax trees (AASTs)*, which are abstract syntax trees (ASTs) with variable and function identifiers removed, preserving only type information.

The primary goal of using InvAASTCluster is to reduce the potentially large set of correct programs into a manageable number of diverse clusters. For each cluster, a single representative is selected, which is the program closest to the cluster centroid in Euclidean space. InvAASTCluster achieves this by extracting structural features from each program's AAST and semantic features from its invariants. These



(a) Part of the AST representation of Listing 5. (b) Our program representation for the snippet presented in Listing 5.

Fig. 3. AST and our graph representation for the small code snippet presented in Listing 5. (a) Part of the AST representation of Listing 5. (b) Our program representation for the snippet presented in Listing 5.

```
1   { // a and b are ints
2     a = a - b;
3   }
```

Listing 5. Small example of a C code block with an expression.

features are combined into Bag-of-Words vector representations, enabling clustering into a small number of clusters. When a buggy program is submitted, InvAASTCluster computes its distance to each cluster representative and selects the closest one as the most suitable candidate to guide the repair process. As illustrated in Fig. 2, if the incorrect program is closest to representative program C, InvAASTCluster returns C to MENTOR for further analysis and repair guidance.

3.2. Variable alignment

Since the problem of program equivalence, that is, determining whether two programs exhibit the same behavior, is undecidable (Rice, 1953), repairing an incorrect program based on a correct implementation is highly challenging. To compare the faulty and correct programs, repair tools must first establish a correspondence between their respective sets of variables. To address this, MENTOR uses a state-of-the-art graph-based program representation to map variables between incorrect and correct implementations. This representation (Orvalho et al., 2023) captures structural information from the abstract syntax trees (ASTs) of imperative programs and leverages graph neural networks (GNNs) to learn how to align variables across program pairs.

Programs are represented as directed graphs to allow bidirectional information flow in the GNN. These graphs are derived from the program's AST and are enriched with semantic information (Orvalho et al., 2023). Fig. 3(a) shows the AST for the code snippet in Listing 5, while Fig. 3(b) shows its corresponding graph representation. Each variable in the program is assigned a unique node, and all occurrences of that variable are connected to it. For example, in Fig. 3(b), the green nodes `a` and `b` represent unique variable nodes.

The graph includes five types of edges:

- *Child edges* (black): connect parent and child nodes in the AST and are bidirectional.
- *Sibling edges* (red dashed): connect each child to its next sibling, capturing argument order.
- *Write edges* (blue dashed): connect an `ID` node to its variable node when the variable is written.
- *Read edges* (green dashed): connect an `ID` node to its variable node when the variable is read.
- *Chronological edges* (yellow dashed): connect `ID` nodes of the same variable to preserve usage order.

All edges are bidirectional, except sibling and chronological edges. This graph structure enables the GNN to propagate information through the program in a semantically meaningful way. After several message-passing rounds through these edges, the GNN generates a numerical vector for each variable node in both programs. Then, scalar products are computed between all pairs of variable vectors from the incorrect and correct programs, followed by a softmax operation to yield the final variable mapping probabilities.

3.3. Fault localization

Effectively identifying the root causes of program failures is essential for generating meaningful repairs (Orvalho et al., 2024b; Jose and Majumdar, 2011b; Lamraoui and Nakajima, 2016). To this end, MENTOR employs CFaults, a state-of-the-art MaxSAT-based fault localization technique. CFaults is designed to identify minimal sets of faulty statements in C programs, even in the presence of multiple bugs. Grounded in Model-Based Diagnosis (MBD) (Reiter, 1987), CFaults encodes multiple failing test cases into a single MaxSAT formula. Each failing test case is treated as an observation, and all are simultaneously represented in a relaxed and unrolled version of the input program. This unified formulation ensures that the identified faults are consistent across all failing test cases.

The approach begins with static analysis to detect simple issues (e.g., uninitialized variables or division by zero). If no issues are found, the program is unrolled and instrumented with relaxation variables that selectively deactivate code statements. The resulting program is then transformed into a CNF formula, and a MaxSAT solver is used to find the minimal set of statements whose relaxation would make all failing test cases pass. By encoding multiple test executions into a single MaxSAT instance, CFaults improves precision in scenarios with multiple faults.

3.4. Counterexample guided automated repair

Large Language Models (LLMs) excel at completing strings, while Formal Methods (FM)-based fault localization excel at identifying buggy parts of a program. Therefore, our approach combines the strengths of both FM and LLMs to enhance automated program repair (APR). Firstly, we employ MaxSAT-based fault localization techniques (see Section 3.3) to rigorously identify the minimal set of buggy parts of a program (Ignatiev et al., 2019; Orvalho et al., 2024b). Afterwards, we leverage LLMs to quickly synthesize the missing parts in the program sketch. Finally, we use a counterexample (i.e., a failing test case from the test suite) to guide LLMs in generating patches that make the synthesized program

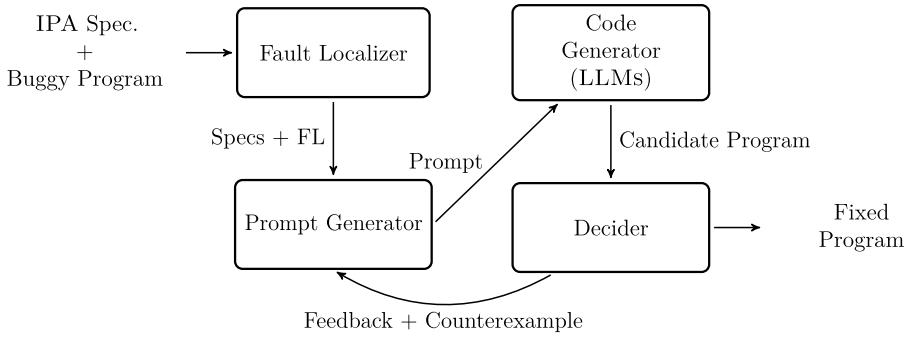


Fig. 4. Counterexample guided automated repair.

compliant with the entire test suite, thus completing the repair. The rationale of our approach follows a Counterexample Guided Inductive Synthesis (CEGIS) (Solar-Lezama et al., 2006) loop to iteratively refine the program. Fig. 4 provides an overview of our APR approach. The input is a buggy program and the specifications for an introductory programming assignment (IPA), including a test suite, a description of the IPA, and the lecturer's reference implementation.

We start by using MaxSAT-based fault localization techniques using all failing test cases to identify the program's minimal set of faulty statements (see Section 3.3). Next, the prompt generator builds a prompt based on the specifications of the IPA and a bug-free program sketch reflecting the localized faults, then feeds this information to the LLM. The LLM generates a program based on the provided prompt. After each iteration, the Decider module evaluates the synthesised program against a test suite, i.e., it checks whether the program passes all the tests in the test suite. If the program is incorrect, a counterexample chosen from the test suite is sent back to the prompt generator, which then provides this counterexample to the LLM to prompt a revised synthesis.

Prompts. The prompts fed to LLMs can contain various types of information related to the IPA. The typical information available in every programming course includes the description of the IPA, the test suite to check the students' submissions corresponding to the IPA's specifications, and the lecturer's reference implementation.

The syntax used in our prompts is similar to that in other works on LLM-driven program repair (Joshi et al., 2023). We have evaluated several types of prompts. Basic prompts are the simplest prompts that can be fed to an LLM without additional computation, including all the programming assignment's basic information. An example of such a prompt is the following: In order to incorporate information about the faults localized in the program using MaxSAT-based fault localization, we utilized two different types of prompts: (1) `FIXME` annotations and (2) program sketches. `FIXME` annotated prompts are prompts where each buggy line identified by the fault localization tool is marked with a `/* FIXME */` comment. These prompts are quite similar to the basic prompt described previously, with the primary differences being the annotations in the buggy program and the first command given to the LLMs, which is modified as follows: In the second type of prompt, to address program repair as a string completion problem, we evaluated the use of prompts where the buggy program is replaced by an incomplete program (program sketch), with each line identified as buggy by our fault localization module replaced by a *hole*. The command given to the LLMs is now to complete the incomplete program. Consequently, the sections 'Buggy Program' and 'Fixed Program' are replaced by 'Incomplete Program' and 'Complete Program', respectively, as follows:

Feedback. If the candidate program generated by the LLM is not compliant with the test suite, this feedback is provided to the LLM in a new message through iterative querying. This new prompt indicates that the LLM's previous suggestion to fix the buggy program was incorrect and provides a counterexample (i.e., an input-output test) where the suggested fixed program produces an incorrect output. Hence, we provide the LLM with a feedback prompt similar to the following:

4. Experimental results

The goal of our evaluation is to answer the following research questions:

- **RQ1.** How effective is MENTOR using LLMs in repairing introductory programming assignments (IPAs) compared to different state-of-the-art symbolic repair approaches?
- **RQ2.** How do different prompt configurations impact the performance of LLMs?
- **RQ3.** How does FM-based fault localization impact LLM-driven APR?
- **RQ4.** What is the performance impact of using a Counterexample Guided approach in LLM-driven APR?
- **RQ5.** How helpful is it to provide a reference implementation for the same IPA to the LLMs?
- **RQ6.** How helpful is it to provide LLMs with a variable mapping between the buggy and a correct implementation of the same IPA?
- **RQ7.** How effective is MENTOR at fixing programs that CLARA fails to repair due to control-flow issues?
- **RQ8.** How does LLM performance vary with different levels of program complexity?
- **RQ9.** How many CEGIS iterations do LLMs typically require to repair programs, and what does this indicate about their repair efficiency?

Experimental setup. All LLMs were run using NVIDIA RTX A4000 graphics cards with 16GB of memory on an Intel(R) Xeon(R) Silver 4130 CPU @ 2.10GHz with 48 CPUs and 128GB RAM. All experiments related to repair tasks were conducted on an Intel(R) Xeon(R) Silver computer with 4210R CPUs @ 2.40GHz, using a CPU memory limit of 10GB and a timeout of 90s. For CLARA and VERIFIX, we set a CPU memory limit of 32GB, as these tools do not utilize GPUs, and a timeout of 90s.

Evaluation benchmark. To evaluate MENTOR, we used C-PACK–IPAS (Orvalho et al., 2024a), which is a set of student programs developed during an introductory programming course in the C programming language. These programs were collected over three distinct lab classes for 25 different programming assignments throughout three academic years. Each lab class focuses on a different topic of the C programming language. The first class deals with integers and input-output operations. Secondly, the second class focuses on loops and chars. Lastly, in the third lab class, students learn to use vectors and strings. Only submissions that compile without any errors were selected. The set of submissions was split into two sets: correct submissions and incorrect submissions. The students' submissions that satisfied a set of input-output test cases for each IPA were considered correct. The submissions that failed at least one input-output test were considered incorrect. C-PACK-IPAS contains 1431 semantically incorrect programs submitted for 25 different IPAs.

````
Fix all semantic bugs in the buggy program below.
Modify the code as little as possible.
Do not provide any explanation.
### Problem Description ###
Write a program that determines and prints the largest of three
integers  given by the user.
### Test Suite
#input:
6 2 1
#output:
6
// The other input-output tests
# Reference Implementation (Do not copy this program) <c> #
```c
int main(){
// Reference Implementation
}
```
### Buggy Program <c> ###
```c
int main(){
// Buggy program from Listing 1
}
```
### Fixed Program <c> ###
```c
````

```
Fix all buggy lines with '/* FIXME */' comments in the buggy
program below.
```

````
Complete all the '@ HOLES N @' in the incomplete program below.
// \ldots
### Incomplete Program <c> ###
// \ldots
### Complete Program <c> ###
```c
````

````
### Feedback ###
Your previous suggestion was incorrect!
Try again.
Code only.
Provide no explanation.
### Counterexample  ###
#input:
6 2 1
#output:
6
### Fixed Program <c> ###
```c
````
````

4.1. Large language models (LLMs)

In our experiments, we used only open-access LLMs available on Hugging Face (HuggingFace, 2024) with approximately 7 billion parameters for three primary reasons. Firstly, closed-access models like ChatGPT are cost-prohibitive and raise concerns over student data privacy. Secondly, models with a very large number of parameters (e.g., 70B) need significant computational resources, such as GPUs with higher RAM capacities, and take longer to generate responses, which is unsuitable for a classroom setting. Thirdly, we used these off-the-shelf LLMs to evaluate the publicly available versions without fine-tuning them. This approach ensures that the LLMs used in this section are available to anyone without investing time and resources into fine-tuning these models. Thus, we evaluated six different LLMs for this study through iterative querying. Three of these models are LLMCs, i.e., LLMs fine-tuned for coding tasks: IBM's Granite (Mishra et al., 2024), Google's CodeGemma (Zhao et al., 2024) and Meta's CodeLlama (Rozière et al., 2023). The other three models are general-purpose LLMs not specifically tailored for coding tasks: Google's Gemma (Mesnard et al., 2024), Meta's Llama3 (latest version of the Llama family Touvron et al., 2023) and Microsoft's Phi3 (Abdin et al., 2024).

We selected specific variants of each model to optimize their performance for our program repair tasks. For Meta's Llama3, we utilized the 8B-parameter instruction-tuned variant. This model is designed to follow instructions more accurately, making it suitable for a range of tasks, including program repair. For CodeLlama, we used the 7B-parameter instruct-tuned version, which is specifically designed for general code synthesis and understanding, making it highly effective for coding tasks. We employed the Granite model with 8B-parameters, fine-tuned to respond to coding-related instructions. For Phi3, we opted for the mini version, which has 3.8B-parameters and a context length of 128K. This smaller model is efficient yet capable of handling extensive context, making it practical for educational settings. For Gemma, we used the 7B-parameter instruction-tuned version, optimized to follow detailed instructions. Lastly, for CodeGemma, we selected the 7B-parameter instruction-tuned variant, designed specifically for code chat and instruction, enhancing its capability to handle programming-related queries and tasks. To fit all LLMs into 16GB GPUs, we used model quantization of 4bit. Moreover, all LLMs were run using Hugging Face's Pipeline architecture. By using these different LLMs, we aimed to balance computational efficiency with the ability to effectively generate and refine code, facilitating a practical APR approach in an educational environment.

4.2. Evaluation

To assess the effectiveness of the program fixes generated by the LLMs under different prompt configurations, we used two key metrics: the number of programs successfully repaired and the quality of the repairs. For assessing the patch quality, we use the *Tree Edit Distance* (TED) (Tai, 1979; Zhang and Shasha, 1989) to compute the distance between the student's buggy program and the fixed program returned by the LLMs. TED computes the structural differences between two Abstract Syntax Trees (ASTs) by calculating the minimum number of edit operations (i.e., insertions, deletions, and substitutions) needed to transform one AST into another.

Based on this metric for measuring program distances, we computed the *distance score*, defined by Eq. (1). This score aims to identify and penalize LLMs that replace the buggy program with the reference implementation rather than fixing it. The distance score is zero when the TED of the original buggy program (T_o) to the program suggested by the LLM (T_f) is the same as the TED of the reference implementation (T_r) to T_o. Otherwise, it penalizes larger fixes than necessary to align the program with the correct implementation.

$$ds(T_f, T_o, T_r) = max\left(0, 1 - \frac{\text{TED}(T_f, T_o)}{\text{TED}(T_r, T_o)}\right) \quad (1)$$

Baseline. We used two state-of-the-art traditional semantic program repair tools for IPAs as baselines: Verifix (Ahmed et al., 2022) and Clara (Gulwani et al., 2018). Verifix employs MaxSMT to align a buggy program with a reference implementation provided by the lecturer, while Clara clusters multiple correct implementations and selects the one that produces the smallest fix when aligned with the buggy program. Both tools require an exact match between the control flow graphs (e.g., branches, loops) and a bijective relationship between the variables; otherwise, they return a structural mismatch error. Verifix was provided with each buggy program, the reference implementation, and a test suite. Clara was given all correct programs from different academic years to generate clusters for each IPA. Within a 90-second time limit, Clara repairs 495 programs (34.6 %), times out without producing a repair on 154 programs (10.8 %), and fails to repair 738 programs (54.7 %). Furthermore, Clara's clustering step was performed offline and is not included in the 90-second time limit. In comparison, Verifix repairs 91 programs (6.3 %), reaches the time limit on 0.6 %, and fails to repair 1338 programs (93.5 %). The main reason for these failures is that both tools rely on structure mismatch errors.

Tables 2 and 3 present the number of programs successfully repaired by each LLM under various configurations. The row labeled `Portfolio` represents the best possible outcomes by selecting the optimal configuration for each program across all LLMs. Meanwhile, `Portfolio` column highlights the best results achieved by a particular LLM across all tested configurations. The configurations yielding the highest success rates for the six evaluated LLMs involve incorporating a reference implementation of the IPA into the prompt (see Table 3). However, rather than genuinely fixing the buggy program, the LLMs often replace it with the reference implementation. To address this, we separately analyzed configurations that include (see Section 4.2.1) and exclude access to a reference implementation.

When no reference implementation is provided (see Table 2), Granite still leads among the LLMs, fixing up to 62.0 % of the programs across all configurations and 48.3 % when using sketches (SK), the IPA description, and a test suite (SK_De-TS). CodeGemma also performs well, achieving up to 59.4 % success in a portfolio approach and showing particular strength in configurations involving sketches (SK). For instance, CodeGemma can repair 48.1 % of the evaluation benchmark using bug-free sketches, IPA description, test suite, and counterexample (SK_De-TS-CE). Configurations incorporating sketches (SK) and FIXME annotations generally yield better results. Including counterexamples (CE), IPA descriptions, and test suites (De-TS) further boosts the success rate across different LLMs. The portfolio approach, which combines the strengths of all LLMs and configurations without using reference implementation, achieves the highest overall success rate, fixing 73.4 % of the programs. This demonstrates that leveraging multiple LLMs together can significantly enhance repair success.

We are now prepared to answer the first four research questions:

A1. All six LLMs used by MENTOR, across different prompt configurations, repair more programs than traditional tools such as Clara and Verifix.

A2. Prompt configurations with FL-based Sketches, IPA description and test suite yield the most successful repair outcomes.

A3. Incorporating FL-based Sketches (or FIXME annotations) allows LLMs to repair more programs than only providing the buggy program.

A4. Employing a Counterexample guided approach significantly improves the accuracy of LLM-driven APR across various configurations.

Table 2
The number of programs fixed by each LLM under various configurations. Row *Portfolio (All LLMs)*, shows the best results across all LLMs for each configuration. Column *Portfolio (All Configurations)* shows the best results for each LLM across all configurations. Mapping abbreviations to configuration names: **CE** - *Counterexample*, **De** - *IPA Description*, **FIXME** - *FIXME Annotations*, **SK** - *Sketches*, **TS** - *Test Suite*.

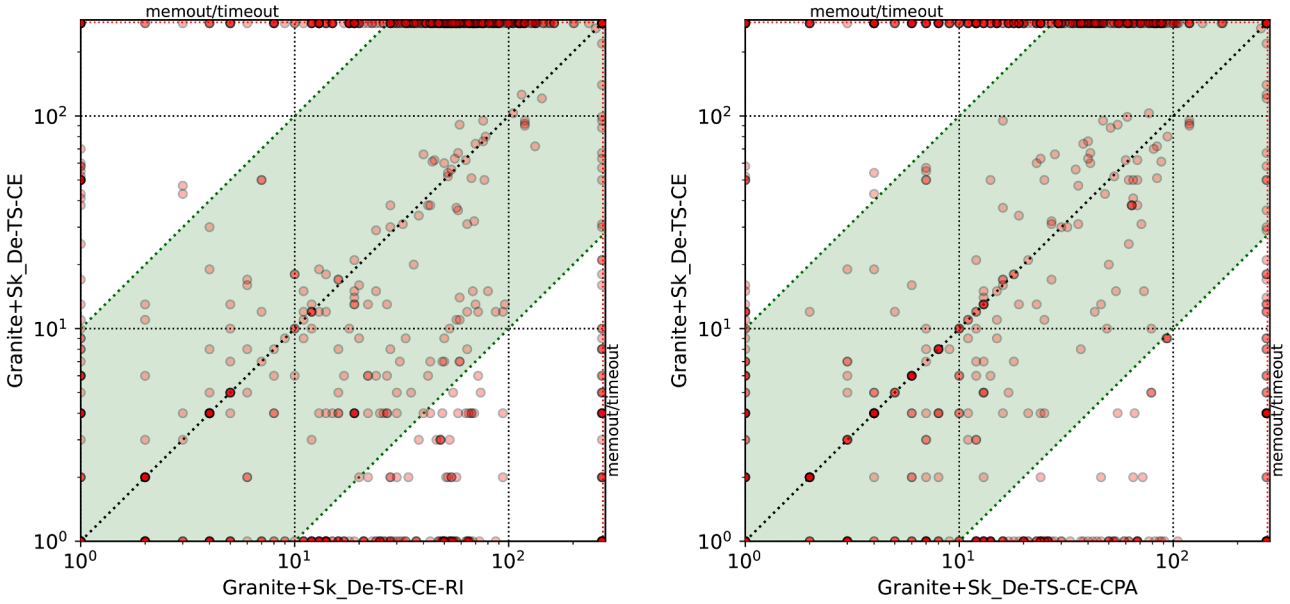
	Prompt Configurations				
LLMs	**CE**	**De**	**De-TS**	**De-TS-CE**	**FIXME**
CodeGemma	451 (31.5 %)	564 (39.4 %)	597 (41.7 %)	606 (42.3 %)	413 (28.9 %)
CodeLlama	447 (31.2 %)	472 (33.0 %)	492 (34.4 %)	500 (34.9 %)	403 (28.2 %)
Gemma	379 (26.5 %)	462 (32.3 %)	496 (34.7 %)	492 (34.4 %)	328 (22.9 %)
Granite	529 (37.0 %)	584 (40.8 %)	626 (43.7 %)	624 (43.6 %)	459 (32.1 %)
Llama3	496 (34.7 %)	533 (37.2 %)	564 (39.4 %)	590 (41.2 %)	400 (28.0 %)
Phi3	367 (25.6 %)	462 (32.3 %)	494 (34.5 %)	489 (34.2 %)	313 (21.9 %)
Portfolio (All LLMs)	732 (51.2 %)	821 (57.4 %)	842 (58.8 %)	846 (59.1 %)	655 (45.8 %)
LLMs	**FIXME_De-CE**	**FIXME_De-TS**	**FIXME_De-TS-CE**	**Sk**	**Sk_De-CE**
CodeGemma	563 (39.3 %)	592 (41.4 %)	601 (42.0 %)	484 (33.8 %)	615 (43.0 %)
CodeLlama	485 (33.9 %)	481 (33.6 %)	463 (32.4 %)	467 (32.6 %)	547 (38.2 %)
Gemma	443 (31.0 %)	446 (31.2 %)	444 (31.0 %)	367 (25.6 %)	524 (36.6 %)
Granite	574 (40.1 %)	566 (39.6 %)	583 (40.7 %)	550 (38.4 %)	653 (45.6 %)
Llama3	531 (37.1 %)	535 (37.4 %)	557 (38.9 %)	434 (30.3 %)	565 (39.5 %)
Phi3	448 (31.3 %)	460 (32.1 %)	474 (33.1 %)	367 (25.6 %)	506 (35.4 %)
Portfolio (All LLMs)	806 (56.3 %)	796 (55.6 %)	820 (57.3 %)	717 (50.1 %)	890 (62.2 %)
LLMs	**Sk_De-TS**	**Sk_De-TS-CE**	**TS**	**Portfolio (All Configurations)**	
CodeGemma	682 (47.7 %)	**688 (48.1 %)**	511 (35.7 %)	850 (59.4 %)	
CodeLlama	**573 (40.0 %)**	561 (39.2 %)	466 (32.6 %)	748 (52.3 %)	
Gemma	532 (37.2 %)	**534 (37.3 %)**	404 (28.2 %)	780 (54.5 %)	
Granite	**691 (48.3 %)**	681 (47.6 %)	577 (40.3 %)	887 (62.0 %)	
Llama3	578 (40.4 %)	**591 (41.3 %)**	505 (35.3 %)	929 (64.9 %)	
Phi3	**547 (38.2 %)**	535 (37.4 %)	400 (28.0 %)	759 (53.0 %)	
Portfolio (All LLMs)	900 (62.9 %)	**907 (63.4 %)**	767 (53.6 %)	1050 (73.4 %)	
Verifix (Ahmed et al., 2022)	90 (6.3 %)				
Clara (Gulwani et al., 2018)	495 (34.6 %)				

Table 3
The number of programs fixed by each LLM under various configurations *with access to a reference implementation* of each IPA. Row *Portfolio (All LLMs)*, shows the best results across all LLMs for each configuration. Column *Portfolio (All Configurations)* shows the best results for each LLM across all configurations. Mapping abbreviations to configuration names: **CE** - *Counterexample*, **CPA** - *Closest Program using AASTs*, **CPIA** - *Closest Program using Invariants + AASTs*, **De** - *IPA Description*, **FIXME** - *FIXME Annotations*, **RI** - *Reference Implementation*, **SK** - *Sketches*, **TS** - *Test Suite*. Entries highlighted in bold correspond to the highest success rates for each LLM.

	Prompt configurations with access to Reference Implementations				
LLMs	**CPA**	**CPIA**	**De-TS-CE-CPA**	**De-TS-CE-RI**	**FIXME_De-TS-CE-CPA**
CodeGemma	505 (35.3 %)	447 (31.2 %)	578 (40.4 %)	576 (40.3 %)	637 (44.5 %)
CodeLlama	532 (37.2 %)	517 (36.1 %)	528 (36.9 %)	525 (36.7 %)	565 (39.5 %)
Gemma	633 (44.2 %)	364 (25.4 %)	595 (41.6 %)	607 (42.4 %)	563 (39.3 %)
Granite	758 (53.0 %)	516 (36.1 %)	773 (54.0 %)	828 (57.9 %)	794 (55.5 %)
Llama3	595 (41.6 %)	464 (32.4 %)	685 (47.9 %)	691 (48.3 %)	657 (45.9 %)
Phi3	465 (32.5 %)	367 (25.6 %)	552 (38.6 %)	444 (31.0 %)	545 (38.1 %)
Portfolio (All LLMs)	1021 (71.3 %)	726 (50.7 %)	1033 (72.2 %)	1046 (73.1 %)	1011 (70.6 %)
LLMs	**FIXME_De-TS-CE-RI**	**FIXME_De-TS-CPA**	**FIXME_De-TS-RI**	**RI**	**Sk_De-TS-CE-CPA**
CodeGemma	638 (44.6 %)	647 (45.2 %)	640 (44.7 %)	445 (31.1 %)	725 (50.7 %)
CodeLlama	609 (42.6 %)	604 (42.2 %)	611 (42.7 %)	455 (31.8 %)	633 (44.2 %)
Gemma	616 (43.0 %)	580 (40.5 %)	655 (45.8 %)	582 (40.7 %)	664 (46.4 %)
Granite	857 (59.9 %)	838 (58.6 %)	882 (61.6 %)	775 (54.2 %)	838 (58.6 %)
Llama3	681 (47.6 %)	662 (46.3 %)	661 (46.2 %)	555 (38.8 %)	725 (50.7 %)
Phi3	492 (34.4 %)	572 (40.0 %)	508 (35.5 %)	358 (25.0 %)	639 (44.7 %)
Portfolio (All LLMs)	1056 (73.8 %)	1036 (72.4 %)	1082 (75.6 %)	1039 (72.6 %)	1050 (73.4 %)
LLMs	**Sk_De-TS-CE-RI**	**Sk_De-TS-CPA**	**Sk_De-TS-RI**	**Portfolio (All Configurations)**	
CodeGemma	739 (51.6 %)	**744 (52.0 %)**	729 (50.9 %)	950 (66.4 %)	
CodeLlama	675 (47.2 %)	673 (47.0 %)	**677 (47.3 %)**	959 (67.0 %)	
Gemma	**732 (51.2 %)**	681 (47.6 %)	720 (50.3 %)	1025 (71.6 %)	
Granite	876 (61.2 %)	881 (61.6 %)	**921 (64.4 %)**	1169 (81.7 %)	
Llama3	730 (51.0 %)	**783 (54.7 %)**	706 (49.3 %)	1073 (75.0 %)	
Phi3	647 (45.2 %)	**661 (46.2 %)**	653 (45.6 %)	959 (67.0 %)	
Portfolio (All LLMs)	1077 (75.3 %)	1080 (75.5 %)	**1089 (76.1 %)**	1218 (85.1 %)	

Table 4
The cumulative distance scores for each program repaired by each LLM across various configurations. Entries highlighted in bold correspond to the highest score for each LLM.

	Metric: **sum(Distance Score)**				
	Prompt Configurations				
LLMs	**FIXME_De-TS**	**FIXME_De-TS-CE**	**FIXME_De-TS-CE-CPA**	**FIXME_De-TS-CE-RI**	**FIXME_De-TS-CPA**
CodeGemma	469.6	479.3	249.2	426.3	250.6
CodeLlama	413.5	403.9	240.4	409.2	258.3
Gemma	284.3	282.2	142.2	264.0	144.1
Granite	463.1	470.6	171.2	298.8	160.4
Llama3	353.4	353.6	175.9	392.7	176.0
Phi3	276.0	291.9	96.8	223.8	96.8
LLMs	**FIXME_De-TS-RI**	**RI**	**Sk**	**Sk_De-CE**	**Sk_De-TS**
CodeGemma	435.7	373.5	421.8	473.8	524.4
CodeLlama	417.3	332.3	421.6	464.8	**477.9**
Gemma	272.5	227.2	280.1	328.6	338.8
Granite	282.7	93.8	479.4	506.2	**539.8**
Llama3	390.6	409.8	354.6	383.1	379.8
Phi3	232.6	203.2	265.1	299.5	**326.5**
LLMs	**Sk_De-TS-CE**	**Sk_De-TS-CE-CPA**	**Sk_De-TS-CE-RI**	**Sk_De-TS-CPA**	**Sk_De-TS-RI**
CodeGemma	**529.5**	249.8	497.3	268.8	484.3
CodeLlama	464.5	251.3	459.0	270.8	452.1
Gemma	**340.3**	156.4	316.2	153.9	307.2
Granite	533.6	172.3	334.5	175.4	349.4
Llama3	384.5	172.7	**423.0**	196.4	407.3
Phi3	321.4	98.2	253.4	97.2	256.6



(a) Reference Implementation. (b) Closest Program using AASTs.

Fig. 5. Comparison of tree edit distances (TED) for GRANITE's repairs when using (x-axis) versus not using (y-axis) correct implementations with configuration Sk_De-TS-CE. (a) Reference Implementation. (b) Closest Program using AASTs.

4.2.1. Correct implementations

Furthermore, in Table 3, we provide the results of LLMs with a reference implementation. The correct implementation can either be the lecturer's reference implementation (RI) for the same IPA or the most similar correct program from a previously submitted student. The closest correct program is determined using Tree Edit Distance (TED) values, computed over the AASTs (CPA) of the programs, or over the AASTs and their invariants (CPIA). The intent was to allow the model to reuse correct code snippets to generate repairs. Results show that including a reference implementation allows for better repair results. However, as mentioned earlier, the LLMs often simply copy the provided reference implementation.

Table 4 presents the sum of the distance scores (see Eq. (1)) for the top-performing LLMs from Tables 2 and 3 across different configurations. This summation aims to penalize LLMs that either copy the provided reference implementation or generate unnecessarily large repairs. For example, GRANITE using configuration Sk_De-TS-CE-RI can repair 876 programs but yields a total distance score of 334.5, whereas using the same configuration without a correct implementation repairs 681 programs resulting in a higher distance score of 533.6.

Fig. 5(a) shows a scatter plot that compares the tree edit distance (TED) of the buggy program to the program fixed by GRANITE with and without a reference implementation, using configuration Sk_De-TS-CE. Each point represents a faulty program, where the x-value (resp. y-value) represents the TED cost of GRANITE' with access to a reference implementation (resp. without it). Points below the diagonal indicate that fixing a program with access to a correct implementation incurs a higher TED cost than fixing it without access. This suggests that while access to a reference implementation enables GRANITE and other LLMs to repair more programs, it often results in larger changes to the student's program than when no correct implementation is given. Similarly, Fig. 5(b) shows a scatter plot that compares the TED cost between the buggy program and the program fixed by GRANITE using the closest correct implementation determined by *anonymized abstract syntax trees* (see Section 3.1), AASTs (CPA), and without using this implementation. This plot shows that while having a correct implementation helps GRANITE repair more programs, it generally results in fewer modifications to the student's code when the CPA is used instead of the lecturer's solution. Moreover, we did not run the same prompt configuration using the closest correct program considering both AASTs and invariants (CPIA) because, as shown in Table 3, this approach did not contribute positively to the repair process of the LLMs.

To answer our fifth research question (RQ5):

A5. Including a reference implementation allows for more repaired programs but with potentially less efficient fixes

4.2.2. Variable mappings

As explained in Section 3.2, mapping variables between two programs is essential for various applications, such as program repair (Orvalho et al., 2023, 2022b; Jiang et al., 2018). In this context, we have evaluated the impact of incorporating variable mappings between the buggy program and a given correct implementation into the prompts used by LLMs. These variable mappings are computed using Graph Neural Networks (GNNs) (Orvalho et al., 2023), which map the set of variables between each correct program and the incorrect submission based on both programs' ASTs.

GNNs. The specific GNN architecture used in this work is the relational graph convolutional neural network (RGCN), which can handle multiple edges or relation types within one graph (Schlichtkrull et al., 2018). The numerical representation of nodes in the graph is updated in the message passing step according to the following equation:

$$\mathbf{x}_i' = \mathbf{\Theta}_{\text{root}} \cdot \mathbf{x}_i + \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{|\mathcal{N}_r(i)|} \mathbf{\Theta}_r \cdot \mathbf{x}_j,$$

where $\mathbf{\Theta}$ are the trainable parameters, $\mathcal{R}$ stands for the different edge types that occur in the graph, and $\mathcal{N}_r$ the neighbouring nodes of the current node i that are connected with the edge type r (PyTorchGeometric, 2022). After each step, we apply Layer Normalization (Ba et al., 2016) followed by a Rectified Linear Unit (ReLU) non-linear function. These GNNs (Orvalho et al., 2023) were trained on the first lab class of C-PACK-IPAS (Orvalho et al., 2024a), with MULTIPAS (Orvalho et al., 2022a) augmenting C-PACK-IPAS by generating pairs of buggy/-correct programs. At training time, since the incorrect program is generated, the mapping between the variables of both programs is known. The network is trained by minimizing the cross entropy loss between the labels (which are categorical integer values indicating the correct mapping) and the values in each corresponding row of the matrix $\mathcal{P}$. As an optimizer, we used the Adam algorithm with its default settings in PyTorch (Kingma and Ba, 2015). The batch size was 1. As there are many different programs generated by the mutation procedures, we took one sample from each mutation for each student. Each network was trained for 20 full passes (epochs) over this dataset while shuffling the order of the training data before each pass. For validation purposes, data corresponding to 20% of the students from the first year of the dataset was kept separate and not trained on. Please refer to Orvalho et al. (2023) for more details about the training and evaluation of the GNNs used in MENTOR.

Table 5 shows the results for LLMs when provided with a correct implementation for the same IPA and a variable mapping between the buggy and correct implementation. The correct implementation can either be the lecturer's reference implementation (RI) for the same IPA or the most similar correct program from a previously submitted student. The closest correct program is determined using Tree Edit Distance (TED) values, computed over the AASTs (CPA) of the programs, or over the AASTs and their invariants (CPIA). As indicated in Table 5, all LLMs, except for GRANITE, are able to repair more programs when they have access to variable mappings between the buggy and correct programs. For instance, CODEGEMMA, using the prompt configuration Sk_De-TS-CE-CPA without variable mappings, fixes 725 programs, while the same configuration plus variable mappings fixes 782 programs, representing an improvement of nearly 8 %.

Table 6 presents the sum of the distance scores (see Eq. (1)) for the LLMs from Table 5 across different prompt configurations. Notably, only LLAMA3's score improves compared to Table 4, increasing from a distance score of 423 to 432.1 in Table 6. This suggests that while access to variable mappings aids LLMs in repairing more programs, it does not significantly enhance the LLMs' distance score in 80 % of the evaluated models.

To answer our sixth research question (RQ6):

A6. Incorporating a variable mapping alongside the reference implementation enables even more programs to be repaired, though it may similarly lead to less efficient fixes.

4.3. Discussion

In response to RQ7, we evaluated the effectiveness of LLMs in repairing programs that CLARA fails to handle due to control-flow issues, which account for 54.7 % of C-PACK-IPAS (738 programs). Table 7 presents these results. Among the best-performing configurations, GRANITE with Sk_De-TS achieved the highest repair rate, successfully fixing 290 programs (37.0 %) in this subset. This highlights GRANITE's strong capability to handle complex program structures where traditional constraint-based tools fail. CODEGEMMA with Sk_De-TS-CE also performed well, repairing 270 programs (34.5 %), demonstrating the advantage of incorporating counterexamples (CE) alongside the Sketches (Sk) configuration. In contrast, models such as LLAMA3 and PHI3 achieved lower success rates, each repairing only 199 programs (25.4 %), suggesting limitations in their ability to generalize and address intricate control-flow issues.

A7. We evaluated 738 programs that CLARA fails to repair due to control-flow issues, representing 54.7 % of C-PACK-IPAS. MENTOR repaired 37.0 % of these programs using GRANITE with Sk_De-TS, followed by CODEGEMMA with Sk_De-TS-CE at 34.5 %, while LLAMA3 and PHI3 performed worse, each repairing only 25.4 %.

Furthermore, to answer RQ8, and to gain deeper insights into MENTOR's performance across varying levels of program complexity, we evaluated the average cyclomatic complexity of each program in C-PACK-IPAS using `lizard` (Lizard, 2024). Table 8 summarizes these findings, divided into quartiles based on cyclomatic complexity. For simpler programs (Q1: 1.0-2.5), CODEGEMMA + Sk_De-TS-CE excelled, achieving a 76.5 % repair rate. However, as program complexity increased (Q3: 3.5-7.0), GRANITE + Sk_De-TS outperformed the other models with a 51.2 % repair rate, underscoring its robustness in tackling moderately complex programs. In the most challenging cases (Q4: 7.0-26), GRANITE retained its lead, repairing 17.1 % of programs. These results suggest that while CODEGEMMA is highly effective for simpler errors, GRANITE exhibits superior adaptability and resilience

Table 5
The number of programs fixed by each LLM under various configurations with access to variable mappings. Row *Portfolio (All LLMs)*, shows the best results across all LLMs for each configuration. Column *Portfolio (All Configurations)* shows the best results for each LLM across all configurations. Mapping abbreviations to configuration names: **CE** - *Counterexample*, **CPA** - *Closest Program using AASTs*, **CPIA** - *Closest Program using Invariants + AASTs*, **De** - *IPA Description*, **FIXME** - *FIXME Annotations*, **RI** - *Reference Implementation*, **SK** - *Sketches*, **TS** - *Test Suite*, **VM** - *Variable Mapping*.

	Prompt configurations with access to Reference Implementations and Variable Mappings				
LLMs	**CPA-VM**	**CPIA-VM**	**De-TS-CE-CPA-VM**	**De-TS-CE-RI-VM**	**FIXME_De-TS-CE-CPA-VM**
CodeGemma	597 (41.7 %)	579 (40.5 %)	651 (45.5 %)	624 (43.6 %)	700 (48.9 %)
CodeLlama	660 (46.1 %)	**699 (48.8 %)**	589 (41.2 %)	568 (39.7 %)	614 (42.9 %)
Gemma	738 (51.6 %)	700 (48.9 %)	675 (47.2 %)	680 (47.5 %)	656 (45.8 %)
Granite	827 (57.8 %)	721 (50.4 %)	821 (57.4 %)	869 (60.7 %)	846 (59.1 %)
Llama3	591 (41.3 %)	565 (39.5 %)	729 (50.9 %)	711 (49.7 %)	689 (48.1 %)
Phi3	552 (38.6 %)	516 (36.1 %)	598 (41.8 %)	555 (38.8 %)	601 (42.0 %)
Portfolio (All LLMs)	1036 (72.4 %)	991 (69.3 %)	1042 (72.8 %)	1070 (74.8 %)	1033 (72.2 %)
LLMs	**FIXME_De-TS-CE-RI-VM**	**RI-VM**	**Sk_De-TS-CE-CPA-VM**	**Sk_De-TS-CE-RI-VM**	**Portfolio (All Configurations)**
CodeGemma	705 (49.3 %)	526 (36.8 %)	**782 (54.6 %)**	780 (54.5 %)	959 (67.0 %)
CodeLlama	618 (43.2 %)	543 (37.9 %)	681 (47.6 %)	677 (47.3 %)	984 (68.8 %)
Gemma	650 (45.4 %)	639 (44.7 %)	756 (52.8 %)	**766 (53.5 %)**	1082 (75.6 %)
Granite	882 (61.6 %)	832 (58.1 %)	901 (63.0 %)	**921 (64.4 %)**	1167 (81.6 %)
Llama3	669 (46.8 %)	559 (39.1 %)	**792 (55.3 %)**	720 (50.3 %)	1060 (74.1 %)
Phi3	531 (37.1 %)	519 (36.3 %)	**691 (48.3 %)**	**691 (48.3 %)**	988 (69.0 %)
Portfolio (All LLMs)	1075 (75.1 %)	1082 (75.6 %)	1078 (75.3 %)	**1093 (76.4 %)**	1210 (84.6 %)

Table 6
The cumulative distance scores for each program successfully repaired by each LLM across various configurations considering variable mappings. Entries highlighted in bold correspond to the highest score for each LLM.

	Metric: **sum(Distance Score)**				
	Prompt Configurations				
LLMs	**CPA-VM**	**CPIA-VM**	**De-TS-CE-CPA-VM**	**De-TS-CE-RI-VM**	**FIXME_De-TS-CE-CPA-VM**
CodeGemma	268.6	382.2	257.6	470.5	275.5
CodeLlama	217.3	351.6	253.9	426.8	231.0
Gemma	137.2	185.3	156.5	243.4	166.1
Granite	78.1	109.4	148.1	283.5	141.8
Llama3	249.9	316.3	199.2	426.0	189.1
Phi3	134.2	182.1	76.4	214.2	75.3
LLMs	**FIXME_De-TS-CE-RI-VM**	**RI-VM**	**Sk_De-TS-CE-CPA-VM**	**Sk_De-TS-CE-RI-VM**	
CodeGemma	475.3	410.1	286.9	**509.5**	
CodeLlama	424.4	372.7	250.0	**455.2**	
Gemma	263.3	195.7	171.5	**306.4**	
Granite	281.3	114.8	165.7	**330.9**	
Llama3	407.0	429.8	196.1	**432.1**	
Phi3	213.0	**252.4**	78.2	247.5	

Table 7
The number of programs repaired by each LLM using their best-performing prompt configuration, specifically on the subset of programs where CLARA fails to repair due to control-flow issues (54.7 % of C-PACK-IPAS).

LLMs	**CodeGemma + Sk_De-TS-CE**	**CodeLlama + Sk_De-TS**	**Gemma + Sk_De-TS-CE**	**Granite + Sk_De-TS**	**Llama3 + Sk_De-TS-CE**	**Phi3 + Sk_De-TS**
#Programs Fixed	270 (34.5 %)	210 (26.8 %)	210 (26.8 %)	**290 (37.0 %)**	199 (25.4 %)	199 (25.4 %)

Table 8
The number of programs repaired by each LLM using their best-performing prompt configuration, considering the average cyclomatic complexity of programs (Lizard, 2024).

Quartiles for Average Cyclomatic Complexity	**CodeGemma + Sk_De-TS-CE**	**CodeLlama + Sk_De-TS + Sk_De-TS**	**Gemma + Sk_De-TS-CE + Sk_De-TS-CE**	**Granite + Sk_De-TS**	**Llama3 + Sk_De-TS-CE**	**Phi3 + Sk_De-TS**	**CLARA**
Q1: 1.0–2.5	**231 (76.5 %)**	178 (58.9 %)	183 (60.6 %)	201 (66.6 %)	217 (71.9 %)	198 (65.6 %)	164 (54.3 %)
Q2: 2.5-3.5	**231 (65.3 %)**	212 (59.9 %)	169 (47.7 %)	**231 (65.3 %)**	192 (54.2 %)	180 (50.9 %)	163 (46.1 %)
Q3: 3.5–7.0	168 (45.3 %)	140 (37.7 %)	129 (34.8 %)	**190 (51.2 %)**	139 (37.5 %)	125 (33.7 %)	124 (33.4 %)
Q4: 7.0–26	58 (14.4 %)	43 (10.6 %)	53 (13.1 %)	**69 (17.1 %)**	43 (10.6 %)	44 (10.9 %)	44 (10.9 %)

when addressing programs of greater complexity.

A8. All evaluated models, including CLARA, face significant challenges when repairing programs with an average cyclomatic complexity above seven. Nevertheless, our LLM-based approach successfully repairs more complex programs than CLARA.

Finally, considering only CEGIS loops where LLMs were able to repair the program within the time limit, the minimum number of iterations is one, the maximum number of iterations to fix a program is seven, and the average number of iterations is 1.14. In 89 % of the cases, the program is repaired on the first attempt. Thus, to answer our final research question (RQ9):

A9. LLMs typically require very few CEGIS iterations to repair programs. On average, only 1.14 iterations are needed, with 89 % of repairs completed on the first attempt and no case requiring more than seven iterations.

5. Related work

Automated Program Repair (APR). Several constraint-based program repair techniques have been proposed to check if a program is semantically correct: clustering-based (Gulwani et al., 2018), implementation-driven (Wang et al., 2018; Ahmed et al., 2022; Hu et al., 2019; Liu et al., 2019b), semantic code search (Ke et al., 2015; Afzal et al., 2019; Stolee et al., 2014; Reiss, 2009), generate-and-validate (Goues et al., 2012; Long and Rinard, 2016), and semantic-based techniques (Nguyen et al., 2013; Mechtaev et al., 2015, 2016). *Clustering-based repair* frameworks (Gulwani et al., 2018; Perry et al., 2019; Wang et al., 2018; Orvalho et al., 2025b) address incorrect student submissions by leveraging a test suite and a pool of N correct submissions for the same IPA. To improve scalability, they first eliminate dynamically equivalent solutions using the provided test suite and clustering techniques, ensuring that only semantically distinct programs are retained. The remaining correct solutions are then grouped into K semantically different clusters ($K << N$), each represented by a cluster exemplar. These representatives serve as the basis for repairing the incorrect submission, reducing redundancy while maintaining coverage of diverse solution strategies. *Implementation-driven repair* tools use one reference implementation to repair a given incorrect submission (Ahmed et al., 2022; Hu et al., 2019; Liu et al., 2019b). AUTOGRADER (Liu et al., 2019b) finds potential path differences between the executions of a student's submission and a reference implementation using symbolic execution. VERIFIX (Ahmed et al., 2022) aligns an incorrect program with the reference solution into an automaton. Then, using that alignment relation and MaxSMT solving, VERIFIX proposes fixes to the incorrect program. Moreover, constraints-based repair tools are developed for specific programming languages and typically support only a subset of that language, based on the focus of the programming course. Extending the supported subset is often hard and time-consuming. Furthermore, *code search methods* (Ke et al., 2015; Afzal et al., 2019; Stolee et al., 2014; Reiss, 2009) form another family of semantic program repair techniques. These approaches rely on a specification (e.g., input-output tests) to query large code repositories and identify fragments that satisfy the specification. Unlike clustering-based or implementation-driven approaches, semantic code search does not rely on the closest correct implementation of the same IPA or on a lecturer-provided reference solution. Instead, it searches across other correct programs to extract code fragments that can be used to repair the incorrect submission. *Generate-and-validate techniques* (Goues et al., 2012; Long and Rinard, 2016) apply predefined mutation operators to the input program to produce multiple candidate solutions, which are then validated against the test suite for correctness. Finally, *semantic-based* approaches rely on symbolic techniques that encode programs into logical formulae for repair. SEMFIX (Nguyen et al., 2013) leverages fault localization to identify suspicious statements and replaces them with symbolic expressions, which are then evaluated against the test suite. DIRECTFIX (Mechtaev et al., 2015) extends this idea by encoding the program as a MaxSMT formula, enabling the synthesis of multi-line repairs. While more powerful than SEMFIX, this approach suffers from scalability issues due to the complexity of solving large formulas. To balance scalability and expressiveness, ANGELIX (Mechtaev et al., 2016) introduces the concept of angelic values, states, paths, and forests, which capture the intended program behavior in symbolic form. An angelic value is the expected value an expression should return to pass a given test. An angelic state is the set of variables that are visible in the scope of the expression being analyzed. An angelic path is encoded as a triple containing the faulty expression and its respective angelic value and state, these paths can be achieved by symbolic execution of programs. Finally, an angelic forest is the set of angelic paths that encode a repair problem. ANGELIX uses these forests to synthesize multi-line fixes.

Large Language Models (LLMs). LLMs trained on code (LLMCs) have demonstrated significant effectiveness in generating program fixes (Joshi et al., 2023; Xia et al., 2023a; Jin et al., 2023; Wei et al., 2023; Fan et al., 2023; Xia et al., 2023b; Bouzenia et al., 2025; Spiess et al., 2025; Ye and Monperrus, 2024). For instance, RING (Joshi et al., 2023) is a multilingual repair engine powered by an LLMC that uses fault localization information from error messages and leverages the few-shot capabilities of LLMCs for code transformation. In the context of APR for programming education, several works have explored the use of LLMs for coding tasks (Zhang et al., 2024; Phung et al., 2023; Liffiton et al., 2023). PyDex Zhang et al. (2024), for example, employs iterative querying with CODEX, an LLMC version of ChatGPT, using test-based few-shot selection and structure-based program chunking to repair syntax and semantic errors in Python assignments. Similarly, CODEHELP (Liffiton et al., 2023) utilizes OpenAI's LLMs to provide textual feedback to students on their programming assignments. However, to the best of our knowledge, no existing work has explored the use of LLMs guided by formula-based fault localization.

Fault localization (FL). Fault localization techniques typically fall into two main families: *spectrum-based (SBFL)* and *formula-based (FBFL)*. SBFL methods (Abreu et al., 2009a; Wong et al., 2010; Naish et al., 2011; Wong et al., 2014, 2016; Abreu et al., 2009b) estimate the likelihood of a statement being faulty based on test coverage information from both passing and failing test executions. While SBFL techniques are generally fast, they may lack precision, as not all identified statements are likely to be the cause of failures (Liu et al., 2019a; Rothenberg and Grumberg, 2020). In contrast, FBFL approaches (Jose and Majumdar, 2011a,b; Lamraoui and Nakajima, 2016; Feser et al., 2015; Lamraoui and Nakajima, 2014; Griesmayer et al., 2006; Wotawa et al., 2012; Xie and Aiken, 2005; Könighofer and Bloem, 2011) are considered exact. FBFL methods encode the fault localization problem into several optimization problems aimed at identifying the minimum number of faulty statements within a program. Typically, these methods perform a MaxSAT call for each failing test, allowing them to individually identify a minimal set of faults for each failing test case rather than simultaneously addressing all failing test cases.

6. Conclusion

In conclusion, MENTOR presents an innovative solution for automated program repair (APR) in the context of introductory programming assignments (IPAs). By addressing the challenges faced by state-of-the-art tools like CLARA and VERIFIX, MENTOR introduces a novel approach that combines advanced techniques for greater flexibility in repairing student submissions. Unlike CLARA, which requires strict structural alignment between faulty and correct programs, MENTOR's LLM-based approach enables flexible repairs without strict structural alignment. Furthermore, MENTOR employs a clustering-based strategy that takes advantage of previously correct submissions to repair programs with different control flow graphs. In addition, MENTOR uses Graph

Neural Networks (GNNs) to map variables between programs more effectively in its variable alignment module, and its fault localization module, CFAULTS is based on Maximum Satisfiability (MaxSAT) techniques, which allows it to pinpoint buggy code with precision.

One of MENTOR's key innovations is the combination this MaxSAT-based fault localization with a program fixer module that integrates Formal Methods (FM) and Large Language Models (LLMs), MENTOR iteratively refines student programs, leading to smaller, more precise fixes. This hybrid approach leverages the strengths of both FM, which excels at fault localization, and LLMs, which are adept at generating code patches, exceeding the capabilities of symbolic repair tools alone.

Experimental results on C-PACK-IPAS demonstrate MENTOR's superior performance, with repair success rates ranging from 37.3 % to 64.4 %, depending on the prompt configuration and LLM used. This marks a significant improvement over the state-of-the-art.

To conclude, MENTOR aims to support both students and lecturers by offering automated personalized feedback in IPAS. For lecturers, MENTOR simplifies the evaluation process and allows programming courses to scale to larger enrollments, as the need for direct personalized feedback from the lecturer is significantly reduced. For students, MENTOR can improve their self-learning experience by providing targeted feedback on syntactic and semantic errors in a shorter time frame.

Currently, MENTOR provides personalized feedback to programmers by offering the repaired program generated by the program fixer or by highlighting bugs in their code. For future work, we propose using the repaired program to provide more detailed feedback, rather than just highlighting errors. This could include providing a sketch of the repaired program as a hint to guide students through fixing their buggy code. Furthermore, future work includes extending MENTOR to programming languages other than C. Moreover, we plan to conduct a user study of MENTOR to gather qualitative insights on its practical utility in educational settings.

CRediT authorship contribution statement

Pedro Orvalho: Writing –original draft, Visualization, Validation, Software, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Mikoláš Janota:** Validation, Supervision, Investigation; **Vasco Manquinho:** Writing –review & editing, Supervision, Resources, Project administration, Funding acquisition.

Data availability

The data and code will be made publicly available upon acceptance of the paper.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Pedro Orvalho reports financial support was provided by European Research Council. Vasco Manquinho reports financial support was provided by Foundation for Science and Technology. Mikolas Janota reports financial support was provided by Ministry of Education Youth and Sports of the Czech Republic. Pedro Orvalho reports financial support was provided by Foundation for Science and Technology. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

PO acknowledges partial support from the ERC under the European Union's Horizon 2020 research and innovation programme (FUN2MODEL, grant agreement No. 834115). This work was also supported by Portuguese national funds through FCT, under projects UID/50021/2025, UID/PRR/50021/2025, and 2023.14280.PEX (DOI: 10.54499/2023.14280.PEX) and grant SFRH/-BD/07724/2020 (DOI: 10.54499/2020.07724.BD). This work was also supported by the MEYS within the program ERC CZ under the project POSTMAN no. LL1902 and co-funded by the EU under the project *ROBOPROX* (reg. no. CZ.02.01.01/00/22_008/0004590).

References

Abdin, M.I., Jacobs, S.A., Awan, A.A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahree, A., Bakhtiari, A., Behl, H.S., Benhaim, A., Bilenko, M., Bjorck, J., Bubeck, S., Cai, M., Mendes, C. C.T., Chen, W., Chaudhary, V., Chopra, P., Giorno, A.D., de Rosa, G., Dixon, M., Eldan, R., Iter, D., Garg, A., Goswami, A., Gunasekar, S., Haider, E., Hao, J., Hewett, R.J., Huynh, J., Javaheripi, M., Jin, X., Kauffmann, P., Karampatziakis, N., Kim, D., Khademi, M., Kurilenko, L., Lee, J.R., Lee, Y.T., Li, Y., Liang, C., Liu, W., Lin, E., Lin, Z., Madan, P., Mitra, A., Modi, H., Nguyen, A., Norick, B., Patra, B., Perez-Becker, D., Portet, T., Pryzant, R., Qin, H., Radmilac, M., Rosset, C., Roy, S., Ruwase, O., Saarikivi, O., Saied, A., Salim, A., Santacroce, M., Shah, S., Shang, N., Sharma, H., Song, X., Tanaka, M., Wang, X., Ward, R., Wang, G., Witte, P., Wyatt, M., Xu, C., Xu, J., Yadav, S., Yang, F., Yang, Z., Yu, D., Zhang, C., Zhang, C., Zhang, J., Zhang, L.L., Zhang, Y., Zhang, Y., Zhang, Y., Zhou, X., 2024. Phi-3 technical report: a highly capable language model locally on your phone. CoRR. https://doi.org/10.48550/arXiv.2404.14219.

Abreu, R., Zoeteweij, P., van Gemund, A. J.C., 2009a. Spectrum-based multiple fault localization. In: ASE 2009, 24th IEEE/ACM International Conference on Automated Software Engineering, Auckland, New Zealand, November 16–20, 2009. IEEE Computer Society, pp. 88–99. https://doi.org/10.1109/ASE.2009.25.

Abreu, R., Zoeteweij, P., Golsteijn, R., van Gemund, A. J.C., 2009b. A practical evaluation of spectrum-based fault localization. J. Syst. Softw. 82 (11), 1780–1792. https://doi.org/10.1016/j.jss.2009.06.035.

Afzal, A., Motwani, M., Stolee, K.T., Brun, Y., Goues, C.L., 2019. Sosrepair: expressive semantic search for real-world program repair. IEEE Trans. Softw. Eng. 47 (10), 2162–2181. https://doi.org/10.1109/TSE.2019.2944914

Ahmed, U.Z., Fan, Z., Yi, J., Al-Bataineh, O.I., Roychoudhury, A., 2022. Verifix: verified repair of programming assignments. ACM Trans. Softw. Eng. Methodol. https://doi.org/10.1145/3510418

Aho, A.V., Sethi, R., Ullman, J.D., 1986. Compilers: Principles, Techniques, and Tools. Addison-Wesley series in computer science / World student series edition, Addison-Wesley.

Allen, F.E., 1970. Control flow analysis. In: Northcote, R.S. (Ed.), Proceedings of a Symposium on Compiler Optimization, Urbana-Champaign, Illinois, USA, July 27–28, 1970. ACM, pp. 1–19.

Ba, L.J., Kiros, J.R., Hinton, G.E., 2016. Layer normalization. CoRR http://arxiv.org/abs/1607.06450.

Bacchus, F., Järvisalo, M., Martins, R., 2021. Maximum satisfiability. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. IOS Press. chapter 24, pp. 929–991.

Bhatia, S., Kohli, P., Singh, R., 2018. Neuro-symbolic program corrector for introductory programming assignments. In: ICSE 2018. ACM, pp. 60–70.

Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), 2009. Handbook of Satisfiability. Vol. 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press.

Bouzenia, I., Devanbu, P.T., Pradel, M., 2025. Repairagent: an autonomous, LLM-based agent for program repair. In: 47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26–May 6, 2025. IEEE, pp. 2188–2200. https://doi.org/10.1109/ICSE55347.2025.00157.

Contractor, M.R., Rivero, C.R., 2022. Improving program matching to automatically repair introductory programs. In: Crossley, S., Popescu, E. (Eds.), Intelligent Tutoring Systems. Springer International Publishing, Cham, pp. 323–335.

Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., Tan, S.H., 2023. Automated repair of programs from large language models. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023. IEEE, pp. 1469–1481. https://doi.org/10.1109/ICSE48619.2023.00128.

Ferrante, J., Ottenstein, K.J., Warren, J.D., 1987. The program dependence graph and its use in optimization. ACM Trans. Program. Lang. Syst. 9 (3), 319–349. https://doi.org/10.1145/24039.24041.

Feser, J.K., Chaudhuri, S., Dillig, I., 2015. Synthesizing data structure transformations from input-output examples. In: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15–17, 2015, pp. 229–239.

Goues, C.L., Nguyen, T., Forrest, S., Weimer, W., 2012. Genprog: a generic method for automatic software repair. IEEE Trans. Softw. Eng. 38 (1), 54–72.

Griesmayer, A., Staber, S., Bloem, R., 2006. Automated fault localization for c programs. In: Bloem, R., Roveri, M., Somenzi, F. (Eds.), Proceedings of the Workshop on Verification and Debugging, v&D@FLoC 2006, Seattle, WA, USA, August 21, 2006. Elsevier, pp. 95–111. https://doi.org/10.1016/j.entcs.2006.12.032.

Gulwani, S., Radicek, I., Zuleger, F., 2018. Automated clustering and program repair for introductory programming assignments. In: PLDI 2018. ACM, pp. 465–480.

Gupta, R., Kanade, A., Shevade, S.K., 2019. Deep reinforcement learning for syntactic error repair in student programs. In: The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019. AAAI Press, pp. 930–937.

Gupta, R., Pal, S., Kanade, A., Shevade, S.K., 2017. Deepfix: fixing common c language errors by deep learning. In: Singh, S.P., Markovitch, S. (Eds.), AAAI 2017. AAAI Press, pp. 1345–1351.

Hu, Y., Ahmed, U.Z., Mechtaev, S., Leong, B., Roychoudhury, A., 2019. Re-factoring based program repair applied to programming assignments. In: 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11–15, 2019. IEEE, pp. 388–398.

HuggingFace, 2024. https://huggingface.co. [Online; accessed 1-July-2024].

Ignatiev, A., Morgado, A., Weissenbacher, G., Marques-Silva, J., 2019. Model-based diagnosis with multiple observations. In: Kraus, S. (Ed.), Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10–16, 2019. ijcai.org, pp. 1108–1115. https://doi.org/10.24963/ijcai.2019/155. https://doi.org/10.24963/IJCAI.2019/155.

Jiang, J., Xiong, Y., Zhang, H., Gao, Q., Chen, X., 2018. Shaping program repair space with existing patches and similar code. In: Tip, F., Bodden, E. (Eds.), Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2018, Amsterdam, The Netherlands, July 16–21, 2018. ACM, pp. 298–309. https://doi.org/10.1145/3213846.3213871

Jin, M., Shahriar, S., Tufano, M., Shi, X., Lu, S., Sundaresan, N., Svyatkovskiy, A., 2023. Inferfix: end-to-end program repair with LLMs. In: Chandra, S., Blincoe, K., Tonella, P. (Eds.), Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3–9, 2023. ACM, pp. 1646–1656. https://doi.org/10.1145/3611643.3613892.

Jose, M., Majumdar, R., 2011a. Bug-assist: assisting fault localization in ANSI-C programs. In: Gopalakrishnan, G., Qadeer, S. (Eds.), Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14–20, 2011. Proceedings. Springer, pp. 504–509. https://doi.org/10.1007/978-3-642-22110-1_40.

Jose, M., Majumdar, R., 2011b. Cause clue clauses: error localization using maximum satisfiability. In: Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011. ACM, pp. 437–446.

Joshi, H., Sánchez, J. P.C., Gulwani, S., Le, V., Verbruggen, G., Radicek, I., 2023. Repair is nearly generation: multilingual program repair with LLMs. In: Williams, B., Chen, Y., Neville, J. (Eds.), Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7–14, 2023. AAAI Press, pp. 5131–5140. https://doi.org/10.1609/aaai.v37i4.25642.

Ke, Y., Stolee, K.T., Goues, C.L., Brun, Y., 2015. Repairing programs with semantic code search (t). In: Cohen, M.B., Grunske, L., Whalen, M. (Eds.), 30th IEEE/ACM International Conference on Automated Software Engineering, ASE 2015. IEEE Computer Society, pp. 295–306.

Kingma, D.P., Ba, J., 2015. Adam: a method for stochastic optimization. In: 3rd International Conference on Learning Representations, ICLR 2015. http://arxiv.org/abs/1412.6980.

Könighofer, R., Bloem, R., 2011. Automated error localization and correction for imperative programs. In: Bjesse, P., Slobodová, A. (Eds.), International Conference on Formal Methods in Computer-Aided Design, FMCAD '11, Austin, TX, USA, October 30–November 02, 2011. FMCAD Inc., pp. 91–100. http://dl.acm.org/citation.cfm?id=2157671.

Lamraoui, S., Nakajima, S., 2014. A formula-based approach for automatic fault localization of imperative programs. In: Merz, S., Pang, J. (Eds.), Formal Methods and Software Engineering - 16th International Conference on Formal Engineering Methods, ICFEM 2014, Luxembourg, Luxembourg, November 3–5, 2014. Proceedings. Springer, pp. 251–266. https://doi.org/10.1007/978-3-319-11737-9_17.

Lamraoui, S., Nakajima, S., 2016. A formula-based approach for automatic fault localization of multi-fault programs. J. Inf. Process. 24 (1), 88–98. https://doi.org/10.2197/ipsjjip.24.88.

Liffiton, M.H., Sheese, B.E., Savelka, J., Denny, P., 2023. Codehelp: using large language models with guardrails for scalable support in programming classes. In: Mühling, A., Jormanainen, I. (Eds.), Proceedings of the 23rd Koli Calling International Conference on Computing Education Research, Koli Calling 2023, Koli, Finland, November 13–18, 2023. ACM, pp. 8:1–8:11. https://doi.org/10.1145/3631802.3631830.

Liu, K., Koyuncu, A., Bissyandé, T.F., Kim, D., Klein, J., Le Traon, Y., 2019a. You cannot fix what you cannot find! an investigation of fault localization bias in benchmarking automated program repair systems. In: 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST). IEEE, pp. 102–113.

Liu, X., Wang, S., Wang, P., Wu, D., 2019b. Automatic grading of programming assignments: an approach based on formal semantics. In: Beecham, S., Damian, D.E. (Eds.), Proceedings of the 41st International Conference on Software Engineering: Software Engineering Education and Training, ICSE (SEET) 2019. IEEE / ACM, pp. 126–137.

Lizard, 2024. Lizard: A simple code complexity analyser. https://github.com/terryyin/lizard. Accessed: 2024-05-01.

Long, F., Rinard, M., 2016. Automatic patch generation by learning correct code. In: Bodík, R., Majumdar, R. (Eds.), Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20–22, 2016. ACM, pp. 298–312. https://doi.org/10.1145/2837614.2837617

Mechtaev, S., Yi, J., Roychoudhury, A., 2015. Directfix: looking for simple program repairs. In: Bertolino, A., Canfora, G., Elbaum, S.G. (Eds.), 37th IEEE/ACM International Conference on Software Engineering, ICSE 2015. IEEE Computer Society, pp. 448–458.

Mechtaev, S., Yi, J., Roychoudhury, A., 2016. Angelix: scalable multiline program patch synthesis via symbolic analysis. In: Dillon, L.K., Visser, W., Williams, L.A. (Eds.), ICSE 2016. ACM, pp. 691–701.

Mesbah, A., Rice, A., Johnston, E., Glorioso, N., Aftandilian, E., 2019. Deepdelta: learning to repair compilation errors. In: ESEC/SIGSOFT FSE 2019. ACM, pp. 925–936.

Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M.S., Love, J., Tafti, P., Hussenot, L., Chowdhery, A., Roberts, A., Barua, A., Botev, A., Castro-Ros, A., Slone, A., Héliou, A., Tacchetti, A., Bulanova, A., Paterson, A., Tsai, B., Shahriari, B., Lan, C.L., Choquette-Choo, C.A., Crepy, C., Cer, D., Ippolito, D., Reid, D., Buchatskaya, E., Ni, E., Noland, E., Yan, G., Tucker, G., Muraru, G., Rozhdestvenskiy, G., Michalewski, H., Tenney, I., Grishchenko, I., Austin, J., Keeling, J., Labanowski, J., Lespiau, J., Stanway, J., Brennan, J., Chen, J., Ferret, J., Chiu, J., et al., 2024. Gemma: open models based on gemini research and technology. CoRR https://doi.org/10.48550/arXiv.2403.08295.

Mishra, M., Stallone, M., Zhang, G., Shen, Y., Prasad, A., Soria, A.M., Merler, M., Selvam, P., Surendran, S., Singh, S., Sethi, M., Dang, X., Li, P., Wu, K., Zawad, S., Coleman, A., White, M., Lewis, M., Pavuluri, R., Koyfman, Y., Lublinsky, B., de Bayser, M., Abdelaziz, I., Basu, K., Agarwal, M., Zhou, Y., Johnson, C., Goyal, A., Patel, H., Shah, S.Y., Zerfos, P., Ludwig, H., Munawar, A., Crouse, M., Kapanipathi, P., Salaria, S., Calio, B., Wen, S., Seelam, S., Belgodere, B., Fonseca, C.A., Singhee, A., Desai, N., Cox, D.D., Puri, R., Panda, R., 2024. Granite code models: a family of open foundation models for code intelligence. CoRR https://doi.org/10.48550/arXiv.2405.04324.

Naish, L., Lee, H.J., Ramamohanarao, K., 2011. A model for spectra-based software diagnosis. ACM Trans. Softw. Eng. Methodol. 20 (3), 11:1–11:32. https://doi.org/10.1145/2000791.2000795

Nguyen, H. D.T., Qi, D., Roychoudhury, A., Chandra, S., 2013. Semfix: program repair via semantic analysis. In: Notkin, D., Cheng, B. H.C., Pohl, K. (Eds.), 35th International Conference on Software Engineering, ICSE '13. IEEE Computer Society, pp. 772–781.

Orvalho, P., Janota, M., Manquinho, V., 2024a. C-pack of IPAs: a C90 program benchmark of introductory programming assignments. In: 2024 IEEE/ACM International Workshop on Automated Program Repair (APR). ACM, pp. 14–21. https://doi.org/10.1145/3643788.3648010

Orvalho, P., Janota, M., Manquinho, V., 2024b. CFaults: model-based diagnosis for fault localization in c programs with multiple test cases. In: Formal Methods - 26th International Symposium, FM 2024, Milan, Italy, 2024, Proceedings. Springer Nature Switzerland, Cham, pp. 463–481. https://doi.org/10.1007/978-3-031-71162-6_24

Orvalho, P., Janota, M., Manquinho, V., 2024c. GitSEED: a git-backed automated assessment tool for software engineering and programming education. In: Proceedings of the 2024 ACM Virtual Global Computing Education Conference, SIGCSE Virtual 2024. Vol. 1. https://doi.org/10.1145/3649165.3690106.

Orvalho, P., Janota, M., Manquinho, V., 2025a. Counterexample guided program repair using zero-shot learning and maxSAT-based fault localization. In: In the 39th Annual AAAI Conference on Artificial Intelligence, AAAI 2025.

Orvalho, P., Janota, M., Manquinho, V., 2025b. InvAASTCluster: on applying invariant-Based program clustering to introductory programming assignments. J. Syst. Softw. 230, 112481. https://doi.org/10.1016/J.JSS.2025.112481.

Orvalho, P., Janota, M., Manquinho, V., 2025c. MENTOR: fixing introductory programming assignments with formula-based fault localization and LLM-driven program repair. https://zenodo.org/records/15678692. https://doi.org/10.5281/zenodo.15678691.

Orvalho, P., Janota, M., Manquinho, V.M., 2022a. MultIPAs: applying program transformations to introductory programming assignments for data augmentation. In: Roychoudhury, A., Cadar, C., Kim, M. (Eds.), Proceedings of the 30Th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022. ACM, pp. 1657–1661. https://doi.org/10.1145/3540250.3558931.

Orvalho, P., Piepenbrock, J., Janota, M., Manquinho, V., 2022b. Project proposal: learning variable mappings to repair programs. In: 7th Conference on Artificial Intelligence and Theorem Proving, AITP.

Orvalho, P., Piepenbrock, J., Janota, M., Manquinho, V.M., 2023. Graph neural networks for mapping variables between programs. In: ECAI 2023 - 26th European Conference on Artificial Intelligence. IOS Press, Poland, pp. 1811–1818. https://doi.org/10.3233/FAIA230468. https://doi.org/10.3233/FAIA230468.

Orvalho, P., Terra-Neves, M., Ventura, M., Martins, R., Manquinho, V.M., 2019. Encodings for enumeration-based program synthesis. In: Principles and Practice of Constraint Programming - 25th International Conference, CP 2019, Stamford, CT, USA, September 30–October 4, 2019, Proceedings, pp. 583–599.

Orvalho, P., Terra-Neves, M., Ventura, M., Martins, R., Manquinho, V.M., 2020. SQUARES : a SQL synthesizer using query reverse engineering. Proc. VLDB Endow. 13 (12), 2853–2856. http://www.vldb.org/pvldb/vol13/p2853-orvalho.pdf.

Perry, D.M., Kim, D., Samanta, R., Zhang, X., 2019. Semcluster: clustering of imperative programming assignments based on quantitative semantic features. In: McKinley, K.S., Fisher, K. (Eds.), Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22–26, 2019. ACM, pp. 860–873.

Phung, T., Cambronero, J., Gulwani, S., Kohn, T., Majumdar, R., Singla, A., Soares, G., 2023. Generating high-precision feedback for programming syntax errors using large language models. In: Feng, M., Käser, T., Talukdar, P.P., Agrawal, R., Narahari, Y., Pechenizkiy, M. (Eds.), Proceedings of the 16th International Conference on Educational Data Mining, EDM 2023, Bengaluru, India, July 11–14, 2023. International Educational Data Mining Society. https://educationaldatamining.org/2023.EDM-short-papers.37/index.html.

PyTorchGeometric, 2022. Documentation. https://pytorch-geometric.readthedocs.io/en/latest/modules/nn.html#torch_geometric.nn.conv.RGCNConv. Accessed 2022-08-12.

Reiss, S.P., 2009. Semantics-based code search. In: 31st International Conference on Software Engineering, ICSE 2009, May 16–24, 2009, Vancouver, Canada, Proceedings. IEEE, pp. 243–253. https://doi.org/10.1109/ICSE.2009.5070525.

Reiter, R., 1987. A theory of diagnosis from first principles. Artif. Intell. 32 (1), 57–95. https://doi.org/10.1016/0004-3702(87)90062-2.

Rice, H.G., 1953. Classes of recursively enumerable sets and their decision problems. Trans. Am. Math. Soc. 74 (2), 358–366.

Rolim, R., Soares, G., D'Antoni, L., Polozov, O., Gulwani, S., Gheyi, R., Suzuki, R., Hartmann, B., 2017. Learning syntactic program transformations from examples. In: Uchitel, S., Orso, A., Robillard, M.P. (Eds.), ICSE 2017. IEEE / ACM, pp. 404–415.

Rothenberg, B., Grumberg, O., 2020. Must fault localization for program repair. In: Lahiri, S.K., Wang, C. (Eds.), Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II. Springer, pp. 658–680. https://doi.org/10.1007/978-3-030-53291-8_33.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton-Ferrer, C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G., 2023. Code llama: open foundation models for code. CoRR. https://doi.org/10.48550/arXiv.2308.12950.

Schlichtkrull, M.S., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M., 2018. Modeling relational data with graph convolutional networks. In: The Semantic Web - 15th International Conference, ESWC 2018. Springer, pp. 593–607. https://doi.org/10.1007/978-3-319-93417-4_38.

Silva, P.d., 2019. SQUARES : A SQL Synthesizer Using Query Reverse Engineering. Master's thesis. Instituto Superior Técnico, Lisboa, Portugal.

da Silva, P. M. O.M., 2025. MENTOR: Automated Feedback for Introductory Programming Exercises. Ph.D. thesis. INSTITUTO SUPERIOR TÉCNICO.

Solar-Lezama, A., Tancau, L., Bodík, R., Seshia, S.A., Saraswat, V.A., 2006. Combinatorial sketching for finite programs. In: Shen, J.P., Martonosi, M. (Eds.), International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 404–415.

Spiess, C., Gros, D., Pai, K.S., Pradel, M., Rabin, M. R.I., Alipour, A., Jha, S., Devanbu, P., Ahmed, T., 2025. Calibration and correctness of language models for code. In: 47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26–May 6, 2025. IEEE, pp. 540–552. https://doi.org/10.1109/ICSE55347.2025.00040.

Stolee, K.T., Elbaum, S.G., Dobos, D., 2014. Solving the search for source code. ACM Trans. Softw. Eng. Methodol. 23 (3), 26:1–26:45. https://doi.org/10.1145/2581377.

Tai, K., 1979. The tree-to-tree correction problem. J. ACM 26 (3), 422–433. https://doi.org/10.1145/322139.322143.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G., 2023. Llama: open and efficient foundation language models. CoRR. https://doi.org/10.48550/arXiv.2302.13971.

Wang, K., Singh, R., Su, Z., 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In: PLDI 2018. ACM, pp. 481–495.

Wei, Y., Xia, C.S., Zhang, L., 2023. Copiloting the copilots: fusing large language models with completion engines for automated program repair. In: Chandra, S., Blincoe, K., Tonella, P. (Eds.), Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3–9, 2023. ACM, pp. 172–184. https://doi.org/10.1145/3611643.3616271.

Wong, W.E., Debroy, V., Choi, B., 2010. A family of code coverage-based heuristics for effective fault localization. J. Syst. Softw. 83 (2), 188–208. https://doi.org/10.1016/j.jss.2009.09.037.

Wong, W.E., Debroy, V., Gao, R., Li, Y., 2014. The DStar method for effective software fault localization. IEEE Trans. Reliab. 63 (1), 290–308. https://doi.org/10.1109/TR.2013.2285319.

Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F., 2016. A survey on software fault localization. IEEE Trans. Software Eng. 42 (8), 707–740. https://doi.org/10.1109/TSE.2016.2521368.

Wotawa, F., Nica, M., Moraru, I., 2012. Automated debugging based on a constraint model of the program and a test case. J. Log. Algebraic Methods Program. 81 (4), 390–407. https://doi.org/10.1016/j.jlap.2012.03.002.

Xia, C.S., Ding, Y., Zhang, L., 2023a. The plastic surgery hypothesis in the era of large language models. In: 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11–15, 2023. IEEE, pp. 522–534. https://doi.org/10.1109/ASE56229.2023.00047.

Xia, C.S., Wei, Y., Zhang, L., 2023b. Automated program repair in the era of large pre-trained language models. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023. IEEE, pp. 1482–1494. https://doi.org/10.1109/ICSE48619.2023.00129.

Xia, C.S., Zhang, L., 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In: Roychoudhury, A., Cadar, C., Kim, M. (Eds.), Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022. ACM, pp. 959–971. https://doi.org/10.1145/3540250.3549101.

Xie, Y., Aiken, A., 2005. Scalable error detection using boolean satisfiability. In: Palsberg, J., Abadi, M. (Eds.), Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12–14, 2005. ACM, pp. 351–363. https://doi.org/10.1145/1040305.1040334.

Yasunaga, M., Liang, P., 2020. Graph-based, self-supervised program repair from diagnostic feedback. In: ICML 2020. PMLR, pp. 10799–10808.

Ye, H., Monperrus, M., 2024. ITER: Iterative neural repair for multi-location patches. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14–20, 2024. ACM, pp. 10:1–10:13. https://doi.org/10.1145/3597503.3623337.

Zhang, J., Cambronero, J.P., Gulwani, S., Le, V., Piskac, R., Soares, G., Verbruggen, G., 2024. Pydex: repairing bugs in introductory python assignments using LLMs. Proc. ACM Program. Lang. 8 (OOPSLA1), 1100–1124. https://doi.org/10.1145/3649850.

Zhang, K., Shasha, D.E., 1989. Simple fast algorithms for the editing distance between trees and related problems. SIAM J. Comput. 18 (6), 1245–1262. https://doi.org/10.1137/0218082.

Zhao, H., Hui, J., Howland, J., Nguyen, N., Zuo, S., Hu, A., Choquette-Choo, C.A., Shen, J., Kelley, J., Bansal, K., Vilnis, L., Wirth, M., Michel, P., Choy, P., Joshi, P., Kumar, R., Hashmi, S., Agrawal, S., Gong, Z., Fine, J., Warkentin, T., Hartman, A.J., Ni, B., Korevec, K., Schaefer, K., Huffman, S., 2024. Codegemma: open code models based on gemma. CoRR. https://doi.org/10.48550/arXiv.2406.11409.